Chapter 2 | Worked solutions with Python

Recurrence and Encoder-Decoder Models

These solutions follow all fourteen exercises in the revised chapter. Begin with the distinction between a state and the rule that updates it. Then follow one number through an RNN, open an LSTM cell, and trace how an encoder prepares a decoder. The final exercises test the implementation and interpret a small learning experiment. Calculations retain full precision until display.

The written version contains every explanation. The notebook version also contains executable checks and their recorded outputs. The equations describe the teaching model; the color task is not a test of translation quality.

Open this notebook inside the companion repository with the packages in `requirements-ch02.txt` and an IPython kernel installed. All examples run on a CPU. The training cell runs 750 small updates in a temporary directory, keeping existing checkpoints intact. Exact floating-point results may vary by software version; assertions check the mathematical and computational properties.

In [1]

```
from pathlib import Path
import sys
import math
import numpy as np
import torch

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    if (repo_root / "examples/ch02_recurrence/model.py").is_file():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError("Open the notebook inside transformers-book.")

from examples.ch02_recurrence.data import BOS, EOS, PAD, VOCAB, collate, make_splits, words
from examples.ch02_recurrence.model import (
    RecurrentSeq2Seq, rnn_trace, gated_cell_update, greedy_decode, token_loss,
)
torch.set_num_threads(1)
torch.manual_seed(2026)
np.set_printoptions(precision=9, suppress=True)
```

1. Separate the state from the rule

Both prefixes end in *green blue*. A two-token model receives the same two IDs, so a deterministic forward calculation with the same weights produces the same distribution. It cannot recover whether the omitted first color was red or yellow by training different weights.

An RNN can reach *green* with different preceding states. Those differences can carry through *blue*. This is an opportunity to preserve the first color, not a guarantee that random or trained weights will actually preserve it. The state is a numerical summary, not a copy of the entire prefix.

Quantity	While reading one sequence	Between training updates
Current token embedding	The selected row can change	The table's used rows can be learned
Encoder hidden and cell states	Updated at each token	Reset to zero for the next unrelated source here
Recurrent weights and biases	The same values are reused	The optimizer can change them

For a concrete check, give the first token teaching features 0.2 or 0.8 and keep the last two features fixed at 0.4 and -0.1. These are illustrative features, not numerical meanings assigned to token IDs. A nonzero recurrent weight lets the different first inputs influence the final states.

In [2]

```
left = rnn_trace([0.2, 0.4, -0.1])
right = rnn_trace([0.8, 0.4, -0.1])
assert not np.isclose(left[-1], right[-1])
print("Final state, first feature 0.2:", f"{left[-1]:.6f}")
print("Final state, first feature 0.8:", f"{right[-1]:.6f}")
print("The update weights are identical in these two calculations.")
```

Output

```
Final state, first feature 0.2: 0.129806
Final state, first feature 0.8: 0.209023
The update weights are identical in these two calculations.
```

2. Follow a state by hand

At each position, first form $a_i = 0.5h_{i-1} + x_i$, then calculate $h_i = \tanh(a_i)$. The previous row's full state becomes the next row's input state. Starting at zero gives:

Position	**Input** x_i	**Sum before tanh** a_i	**New state** h_i
1	0.2	0.200000	0.197375
2	0.4	0.498688	0.461084
3	-0.1	0.130542	0.129806

The last input subtracts 0.1, but the old-state contribution is about $0.5(0.4610844448) = 0.2305422224$. Their sum stays positive, so the new state does too. The current input is only one part of the update.

All table entries are rounded for display. Formatting with six decimals does not replace the stored full-precision state. The input weight 1, recurrent weight 0.5, bias 0, and tanh rule are shared across all three positions.

In [3]

```
inputs = np.array([0.2, 0.4, -0.1])
states = rnn_trace(inputs)
expected = [0.197375320224904, 0.461084444830037, 0.129805707303656]
np.testing.assert_allclose(states, expected, atol=1e-14, rtol=0)
previous = 0.0
print("Step  Input       Sum      State")
for step, (value, state) in enumerate(zip(inputs, states), 1):
    print(f"{step:4d} {value:6.2f} {0.5*previous+value:9.6f} {state:9.6f}")
    previous = state
```

Output

```
Step  Input       Sum      State
   1   0.20  0.200000  0.197375
   2   0.40  0.498688  0.461084
   3  -0.10  0.130542  0.129806
```

3. Reverse the order

Reversing the inputs gives the following trace, again starting from zero:

Position	Input	Sum before tanh	New state
1	-0.1	-0.100000	-0.099668
2	0.4	0.350166	0.336523
3	0.2	0.368261	0.352470

The middle input is still 0.4, but it is now combined with a negative preceding state rather than a positive one. Sharing weights means applying the same rule, not supplying that rule with identical arguments. The final state changes from about 0.129806 to 0.352470.

If the recurrent weight is zero, the rule reduces to $h_i = \tanh(x_i)$. Only the last input affects the final state. Reversing a sequence can still change that last input, so the two final states need not agree. The important point is that changing an earlier input while keeping the last one fixed now has no effect on the final state.

In [4]

```
reverse_states = rnn_trace(inputs[::-1])
np.testing.assert_allclose(reverse_states,
    [-0.099667994624956, 0.336522755846160, 0.352470022359831], atol=1e-12)
print("Reversed states:", reverse_states)
for row in ([0.2, 0.4, -0.1], [0.8, 99.0, -0.1]):
    final = rnn_trace(row, recurrent_weight=0)[-1]
    np.testing.assert_allclose(final, np.tanh(-0.1))
    print("No recurrent contribution, final state:", f"{final:.6f}")
```

Output

```
Reversed states: [-0.099667995  0.336522756  0.352470022]
No recurrent contribution, final state: -0.099668
No recurrent contribution, final state: -0.099668
```

4. Keep, write, and expose

Separate the two contributions to the cell before calculating the hidden value: $c_i = f_i(0.5) + i_i(-0.4)$, then $h_i = o_i \tanh(c_i)$. Each changed case starts again from the original values; changes do not accumulate across the rows.

Case	Retained old cell	New write	Cell c_i	Hidden h_i
Original gates	0.4	-0.2	0.2	0.118425
Forget gate 0	0	-0.2	-0.2	-0.118425
Forget gate 1	0.5	-0.2	0.3	0.174788
Input gate 0	0.4	0	0.4	0.227969
Output gate 0	0.4	-0.2	0.2	0

Opening the forget gate fully retains the old contribution, but does not stop the candidate from changing the new cell. Closing the input gate prevents the negative write, so the cell is larger in this example. Closing only the output

gate leaves the cell intact while hiding it from the current exposed state. A zero hidden value therefore does not imply an empty cell.

The input and forget gates are independent multipliers, not complementary probabilities. A candidate can be negative. Values exactly 0 and 1 are idealized gate limits; a sigmoid of a finite real input lies strictly between them.

In [5]

```
cases = [("original", .8, .5, .6), ("forget=0", 0., .5, .6),
         ("forget=1", 1., .5, .6), ("input=0", .8, 0., .6),
         ("output=0", .8, .5, 0.)]
for name, forget, write, expose in cases:
    hidden, cell = gated_cell_update(.5, forget, write, -.4, expose)
    print(f"{name:10s} cell={cell: .6f} hidden={hidden: .6f}")
hidden, cell = gated_cell_update(.5, .8, .5, -.4, 0.)
assert hidden == 0 and np.isclose(cell, .2)
```

Output

```
original   cell= 0.200000 hidden= 0.118425
forget=0   cell=-0.200000 hidden=-0.118425
forget=1   cell= 0.300000 hidden= 0.174788
input=0    cell= 0.400000 hidden= 0.227969
output=0   cell= 0.200000 hidden= 0.000000
```

5. Compare two gradient paths

The two products are

$$0.5^{20} = 0.0000009536743164, \qquad 0.99^{20} = 0.8179069376.$$

For the scalar RNN, each state derivative is $0.5(1 - h_i^2)$. Since $1 - h_i^2 \leq 1$, the magnitude of the derivative across twenty updates is at most 0.5^{20}. It is an upper bound: the tanh derivative generally makes the actual product smaller. The code checks this with twenty inputs of 0.2, comparing the product to a finite-difference estimate that perturbs the initial state.

For the LSTM, 0.99^{20} describes only the direct cell-carry path with all forget gates held fixed at 0.99. About 81.8% of a small perturbation survives along that path. Other routes involve changing gates, candidates, the hidden state, and the loss. They can diminish or amplify the total derivative, and the forget gates need not stay at 0.99. This one path does not guarantee the size of every parameter gradient.

A state value tells us where the computation is. A derivative tells us how much that value would change under a small change elsewhere. They are different quantities: a nonzero final state can have tiny sensitivity to a far earlier state.

In [6]

```
twenty_inputs = [0.2] * 20
trace = rnn_trace(twenty_inputs)
path_derivative = np.prod(0.5 * (1 - trace**2))
epsilon = 1e-3
plus = rnn_trace(twenty_inputs, initial_state=epsilon)[-1]
minus = rnn_trace(twenty_inputs, initial_state=-epsilon)[-1]
finite_difference = (plus - minus) / (2 * epsilon)
np.testing.assert_allclose(path_derivative, finite_difference, rtol=2e-6, atol=1e-12)
assert 0 < path_derivative <= .5**20
```

```
print(f"RNN upper bound:      {.5**20:.12e}")
print(f"Actual path product:  {path_derivative:.12e}")
print(f"Finite difference:    {finite_difference:.12e}")
print(f"LSTM direct path:     {.99**20:.10f}")
print(f"Final RNN state:      {trace[-1]:.6f}")
```

Output

```
RNN upper bound:      9.536743164062e-07
Actual path product:  6.731069220669e-08
Finite difference:    6.731071255928e-08
LSTM direct path:     0.8179069376
Final RNN state:      0.364782
```

6. Pass the right state

Each final state has shape $(1, 5, 64)$: one layer times one direction, five examples, and 64 state coordinates. The final hidden and cell tensors have no source-position axis. In a variable-length batch, each row uses the state after that example's last real source token.

The decoder's initial hidden state receives the encoder's final hidden state; its initial cell receives the final cell. These values carry information about the current sources. The encoder and decoder keep their own distinct weight matrices. Copying states does not copy or tie their parameters.

At the first decoder update, the hidden state and BOS embedding determine the gates and candidate; the cell supplies the old content to retain. The source embedding of the last word alone supplies neither the state pair nor the earlier context. Two sources ending in the same word retrieve the same last-token embedding, but can have different final encoder states. In this model the embedding width is 16, which also differs from the state width 64.

The padding rows deserve a separate distinction: the model stores 42,857 scalar parameter entries, including two 16-entry PAD embedding rows fixed at zero. This count is unrelated to how many state vectors a batch produces.

In [7]

```
torch.manual_seed(2026)
shape_model = RecurrentSeq2Seq().eval()
five = collate([(3,4,5), (6,3), (8,5,7,3), (4,4,5), (3,7,8)])
with torch.no_grad():
    state_pair = shape_model.encode(five.source, five.lengths)
assert all(tuple(state.shape) == (1, 5, 64) for state in state_pair)
assert shape_model.encoder.weight_ih_l0 is not shape_model.decoder.weight_ih_l0
print("Final hidden and cell shapes:", [tuple(s.shape) for s in state_pair])
print("Stored parameter entries:", sum(p.numel() for p in shape_model.parameters()))
print("Fixed PAD embedding entries:", 2 * shape_model.source_embedding.embedding_dim)
```

Output

```
Final hidden and cell shapes: [(1, 5, 64), (1, 5, 64)]
Stored parameter entries: 42857
Fixed PAD embedding entries: 32
```

7. Shift before grading

Reverse the source colors first, then add the boundary tokens. This gives five prediction steps for a four-color source.

Step	Current decoder input	Reference label
1	BOS	yellow
2	yellow	green
3	green	blue
4	blue	red
5	red	EOS

At the step graded against blue, the input is green. Blue becomes an input only at the following step. If generation selected white instead of green at step 2, teacher forcing would still supply green at step 3, whereas generation would supply white. The two step-3 states would therefore receive different embeddings. The label is used to grade a prediction before it is supplied as a later input.

In [8]

```
shifted = collate([(3, 5, 4, 6)])  # red blue green yellow
print("Step  Decoder input  Label")
for t, (current, target) in enumerate(zip(
        shifted.decoder_input[0], shifted.target[0]), 1):
    print(f"{t:4d}  {VOCAB[int(current)]:13s}  {VOCAB[int(target)]}")
assert shifted.decoder_input[0, 2].item() == 4  # green
assert shifted.target[0, 2].item() == 5         # blue
```

Output

```
Step  Decoder input  Label
   1  <bos>          yellow
   2  yellow         green
   3  green          blue
   4  blue           red
   5  red            <eos>
```

8. Count prediction positions

The longest source has five tokens. Adding EOS makes the longest target six labels. With batch size two, embedding width 16, state width 64, and nine vocabulary classes, the shapes are:

Quantity	Shape	Meaning of the axes
Source IDs	$(2, 5)$	batch, stored source position
Source embeddings	$(2, 5, 16)$	batch, position, embedding coordinate
Decoder inputs and labels	$(2, 6)$ each	batch, stored target position
Vocabulary logits	$(2, 6, 9)$	batch, target position, vocabulary class
Final hidden or cell state	$(1, 2, 64)$ each	layer times direction, batch, coordinate

The shorter row has four real labels and two PAD labels. The longer has six real labels. Thus 12 rectangular slots hold only ten real predictions, including two endings. Masking removes the two PAD positions from the loss; it does not remove the PAD vocabulary class from valid softmax rows.

The token-weighted mean of sentence means 0.2 and 0.8 is

$$\frac{4(0.2) + 6(0.8)}{10} = 0.56.$$

The unweighted sentence mean is $(0.2 + 0.8)/2 = 0.5$. The first gives each token equal weight; the second gives each sentence equal weight. State which average you report instead of treating the two as interchangeable.

In [9]

```
length_batch = collate([(3,5,4), (6,4,3,5,8)])
with torch.no_grad():
    logits = shape_model(length_batch.source, length_batch.lengths,
                         length_batch.decoder_input)
print("Source:", tuple(length_batch.source.shape))
print("Source embeddings:", tuple(shape_model.source_embedding(length_batch.source).shape))
print("Decoder inputs and labels:", tuple(length_batch.target.shape))
print("Logits:", tuple(logits.shape))
print("Non-PAD labels:", int((length_batch.target != PAD).sum()))
assert int((length_batch.target != PAD).sum()) == 10
print("Token mean:", (4*.2 + 6*.8)/10, " Sentence mean:", (.2+.8)/2)
```

Output

```
Source: (2, 5)
Source embeddings: (2, 5, 16)
Decoder inputs and labels: (2, 6)
Logits: (2, 6, 9)
Non-PAD labels: 10
Token mean: 0.56  Sentence mean: 0.5
```

9. Grade the ending

These probabilities come from four different distributions, each conditioned on its own correct prefix and the source. They need not sum to one. Within each prediction step, the probabilities over the vocabulary do sum to one.

Quantity	Original EOS probability 0.9	EOS probability reduced to 0.1
Complete reference probability	$0.8(0.6)(0.75)(0.9) = 0.324$	$0.8(0.6)(0.75)(0.1) = 0.036$
Total negative log-probability	1.127011763	3.324236341
Mean over four labels	0.281752941	0.831059085

The increase in mean loss is

$$\frac{-\ln(0.1) + \ln(0.9)}{4} = \frac{\ln 9}{4} \approx 0.549306144.$$

After the last ordinary word, the reference asks the model to end. Assigning less probability to EOS assigns more to continuing incorrectly and lowers the probability of completing the exact reference. Changing EOS therefore

changes the score even when all ordinary-token probabilities stay fixed. The loss grades probabilities, while greedy decoding takes a single highest-scoring choice at each step.

In [10]

```
probabilities = np.array([.8, .6, .75, .9])
weaker_ending = probabilities.copy()
weaker_ending[-1] = .1
for name, values in [("original", probabilities), ("weaker EOS", weaker_ending)]:
    total_loss = -np.log(values).sum()
    print(f"{name:10s} probability={values.prod():.6f} "
          f"total={total_loss:.9f} mean={total_loss/4:.9f}")
increase = -np.log(weaker_ending).mean() + np.log(probabilities).mean()
np.testing.assert_allclose(increase, np.log(9)/4)
print(f"Increase in mean loss: {increase:.9f}")
```

Output

```
original   probability=0.324000 total=1.127011763 mean=0.281752941
weaker EOS probability=0.036000 total=3.324236341 mean=0.831059085
Increase in mean loss: 0.549306144
```

10. Locate the information path

The source embedding of car enters encoder update 3 together with the states produced after the and red. The resulting pair initializes the decoder. The path to the distribution that will be graded against coche is

$$\begin{aligned}\text{embedding of car} &\longrightarrow (h_3^{\text{enc}}, c_3^{\text{enc}}) \longrightarrow (h_0^{\text{dec}}, c_0^{\text{dec}}) \\ &\xrightarrow{\text{input BOS}} (h_1^{\text{dec}}, c_1^{\text{dec}}) \xrightarrow{\text{input el}} (h_2^{\text{dec}}, c_2^{\text{dec}}) \\ &\longrightarrow z_2 \longrightarrow p_2(\text{coche}).\end{aligned}$$

The output layer reads the updated hidden state. The cell affects that hidden state through the LSTM update. In this no-attention model, the decoder cannot directly reread the source embedding or choose an earlier encoder output. Source information reaches it through the transferred pair and its subsequent recurrent updates.

Keeping a representation for every source position would provide additional access. A decoder could consult car while predicting coche, then consult red while predicting rojo. Chapter 3 develops how to learn this access.

The executable check uses the analogous color model, not an English-Spanish translator. It differentiates a second-step target score with respect to the last source color's embedding. A nonzero gradient confirms a computational route through both networks; it does not show that untrained weights have learned a useful translation or memory strategy.

In [11]

```
torch.manual_seed(2026)
path_model = RecurrentSeq2Seq().double().eval()
path_batch = collate([(3,4,5)])
scores = path_model(path_batch.source, path_batch.lengths, path_batch.decoder_input)
scores[0, 1, 4].backward()  # one vocabulary score at target step 2
last_source_gradient = path_model.source_embedding.weight.grad[5].norm().item()
assert last_source_gradient > 0
assert path_model.encoder.weight_ih_l0.grad.norm() > 0
assert path_model.decoder.weight_ih_l0.grad.norm() > 0
print(f"Gradient norm at the last source embedding: {last_source_gradient:.8f}")
```

```
print("The target score depends on the source through the state connection.")
```

Output

```
Gradient norm at the last source embedding: 0.01258011
The target score depends on the source through the state connection.
```

11. Test padding

Start with red green blue and true length 3. Store it once in a row of width 3 and once in a row of width 6 with three extra PAD entries. Both calls should stop the encoder after blue. With fixed weights and the same decoder inputs, packing gives equal final hidden states, cell states, and decoder logits.

Packing uses the true lengths to skip the added recurrent updates. By contrast, directly passing a dense embedding array to the LSTM processes every rectangular position. A zero padding embedding removes the new-input term, but leaves the preceding hidden-state contribution and the biases. These can change gates and update the cell. Zero input is not an instruction to keep the state unchanged.

The code checks every coordinate, rather than only the printed shapes. The first three differences should be zero up to numerical precision. The final two comparisons deliberately omit packing and demonstrate the unwanted change in this seeded model. Source padding and ignored target labels solve different problems: the first protects the source summary; the second keeps storage positions out of the objective.

In [12]

```
torch.manual_seed(2026)
padding_model = RecurrentSeq2Seq().double().eval()
batch = collate([(3,4,5)])
extended = torch.cat([batch.source, torch.full((1,3), PAD, dtype=torch.long)], dim=1)
with torch.no_grad():
    first = padding_model.encode(batch.source, batch.lengths)
    second = padding_model.encode(extended, batch.lengths)
    original_logits = padding_model(batch.source, batch.lengths, batch.decoder_input)
    extended_logits = padding_model(extended, batch.lengths, batch.decoder_input)
    _, dense = padding_model.encoder(padding_model.source_embedding(batch.source))
    _, dense_extended = padding_model.encoder(padding_model.source_embedding(extended))
for name, a, b in [("packed hidden", first[0], second[0]),
                   ("packed cell", first[1], second[1]),
                   ("decoder logits", original_logits, extended_logits)]:
    torch.testing.assert_close(a, b, rtol=0, atol=1e-14)
    print(f"{name:15s} maximum difference: {(a-b).abs().max().item():.3e}")
assert torch.count_nonzero(padding_model.source_embedding(extended)[:,3:]) == 0
for name, a, b in zip(["dense hidden", "dense cell"], dense, dense_extended):
    change = (a-b).abs().max().item()
    assert change > 0
    print(f"{name:15s} maximum difference: {change:.6f}")
```

Output

```
packed hidden   maximum difference: 0.000e+00
packed cell     maximum difference: 0.000e+00
decoder logits  maximum difference: 0.000e+00
dense hidden    maximum difference: 0.167324
dense cell      maximum difference: 0.325213
```

12. Separate scoring from generation

Teacher-forced argmax choices are made at reference positions with correct prefixes. They are not one freely generated sentence. Greedy generation commits to each selected token and supplies it at the next step. The computations can agree up to the first error and then diverge when their inputs differ.

Train the companion model for 750 updates with seed 2026 on one CPU thread. Validation loss, not test performance, chooses the checkpoint. Reload its saved weights and select the first incorrect held-out generation in split order. If a rerun has no incorrect generations, inspect a seeded untrained model for this exercise while keeping the trained scores for Exercise 13. The training cell uses a temporary directory and does not overwrite an existing experiment.

In the checked run, the first incorrect source was green blue black red red red. The complete reference was red red red black blue green EOS. The following table records how the two passes diverged:

Step	Reference	Forced input	Forced choice	Greedy input	Greedy choice
1	red	BOS	red	BOS	red
2	red	red	red	red	red
3	red	red	red	red	red
4	black	red	black	red	black
5	blue	black	green	black	green
6	green	blue	green	green	blue
7	EOS	green	EOS	blue	EOS

Both passes first chose the wrong color at step 5. At step 6, teacher forcing supplied the reference blue; generation supplied its selected green. Ending correctly at step 7 did not repair the swapped colors. A different run may have a different first error; the printed timeline determines what happened in that run. Any generated PAD or BOS before the first EOS is an actual selection and must be retained when grading it.

In [13]

```
from contextlib import redirect_stdout
from io import StringIO
from tempfile import TemporaryDirectory
from examples.ch02_recurrence.run_examples import run_training

training_seed = 2026
with TemporaryDirectory(prefix="ch02-reviewed-solutions-") as directory:
    with redirect_stdout(StringIO()):
        training_run = run_training(steps=750, seed=training_seed, output=Path(directory))
    checkpoint = torch.load(Path(directory)/"best.pt", map_location="cpu", weights_only=True)
trained_model = RecurrentSeq2Seq()
trained_model.load_state_dict(checkpoint["model"])
trained_model.eval()
train_set, validation_set, test_set = make_splits(seed=training_seed)
test_metrics = training_run["test"]
assert checkpoint["selected_step"] == training_run["selected_step"]
print("Environment:", training_run["environment"])
print("Selected update:", training_run["selected_step"])
print("Checkpoint criterion:", training_run["configuration"]["checkpoint_selection"])
```

Output

```
Environment: {'python': '3.12.14', 'torch': '2.15.0', 'numpy': '2.5.4', 'device': 'cpu', 'threads': 1}
Selected update: 600
Checkpoint criterion: validation token NLL
```

The following cell prints every teacher-forced position, including positions after an early EOS choice. The greedy row stops at its first EOS. A missing entry means that the corresponding computation has already ended.

In [14]

```
from itertools import zip_longest

case_index = next((j for j, (source, prediction) in enumerate(zip(
    test_set, test_metrics["predictions"]))
    if prediction != list(reversed(source)) + [EOS]), None)
case_model = trained_model
if case_index is None:
    case_index = 0
    torch.manual_seed(training_seed)
    case_model = RecurrentSeq2Seq().eval()
    print("All trained generations matched; inspecting untrained weights.")
case = collate([test_set[case_index]])
with torch.no_grad():
    forced = case_model(case.source, case.lengths, case.decoder_input).argmax(-1)[0].tolist()
    generated = greedy_decode(case_model, case.source, case.lengths, 7)[0].tolist()
reference = case.target[0].tolist()
greedy_inputs = [BOS] + generated[:-1]
if case_model is trained_model:
    assert generated == test_metrics["predictions"][case_index]
print("Source:", words(case.source[0]))
print("Reference:", words(reference))
print("Generated:", words(generated))
print("Step  Label    Forced-in  Forced-out Greedy-in  Greedy-out")
first_error = None
for t, row in enumerate(zip_longest(reference, case.decoder_input[0].tolist(),
                                    forced, greedy_inputs, generated), 1):
    labels = ["--" if value is None else VOCAB[value] for value in row]
    print(f"{t:4d}  " + " ".join(f"{label:10s}" for label in labels))
    if first_error is None and row[0] != row[4]:
        first_error = t
print("First greedy mismatch:", first_error)
```

Output

```
Source: green blue black red red red
Reference: red red red black blue green <eos>
Generated: red red red black green blue <eos>
Step  Label    Forced-in  Forced-out Greedy-in  Greedy-out
   1  red        <bos>      red        <bos>      red
   2  red        red        red        red        red
   3  red        red        red        red        red
   4  black      red        black      red        black
   5  blue       black      green      black      green
   6  green      blue       green      green      blue
   7  <eos>      green      <eos>      blue       <eos>
First greedy mismatch: 5
```

13. Read a length experiment carefully

The checked run selected update 600 by the lowest validation token loss. It generated 43 of the 48 complete test sequences correctly. Exact match requires every reversed color and the ending, with no extra tokens.

Source length	Training examples	Validation examples	Test examples	Exact test matches	Rate
3	48	12	12	12	100%
4	48	12	12	12	100%
5	48	12	12	12	100%
6	48	12	12	7	58.33%

Every tested length also occurs in training. Complete sequences are disjoint, but their lengths and vocabulary are familiar. These results therefore test new arrangements within the trained length range, not length extrapolation. The concentration of errors at length six is an observation worth exploring; it does not establish a universal capacity limit or identify a single cause.

One prospective extrapolation design is to train only at lengths 3 and 4, select settings and a checkpoint using length 5, and reserve lengths 6 and 7 for the final test. Decide the split and selection rule before inspecting test results, use unique source tuples, and report counts for every group. Validation at length 5 informs selection; test lengths 6 and 7 remain unseen by both fitting and that selection procedure.

Use a common generation cap of at least eight new tokens for this design: seven colors plus EOS. A cap of seven would make an otherwise correct seven-color answer fail solely because it could not emit its ending. This is a proposed experiment, not a claimed measured result. The default `make_splits` balances the same supplied lengths across all three splits; implementing the proposed design requires constructing separate length sets rather than simply changing that shared argument.

In [15]

```
from collections import Counter
train_counts = Counter(map(len, train_set))
validation_counts = Counter(map(len, validation_set))
test_counts = Counter(map(len, test_set))
assert not (set(train_set) & set(validation_set) or set(train_set) & set(test_set)
            or set(validation_set) & set(test_set))
print("Length Train Valid Test Exact Rate")
for length in sorted(test_counts):
    group = test_metrics["by_length"][str(length)]
    exact, count = group["exact_matches"], group["sequences"]
    assert count == test_counts[length] == 12
    print(f"{length:6d} {train_counts[length]:5d} {validation_counts[length]:5d} "
          f"{count:4d} {exact:5d} {exact/count:6.2%}")
print("Total exact:", test_metrics["exact_matches"], "/", test_metrics["sequences"])
print("Teacher-forced correct positions:", test_metrics["teacher_forced_correct_tokens"],
      "/", test_metrics["nonpadding_tokens"])
print("Proposed lengths: train [3,4], validation [5], test [6,7]; common cap 8.")
```

Output

```
Length Train Valid Test Exact Rate
     3    48    12   12    12 100.00%
     4    48    12   12    12 100.00%
     5    48    12   12    12 100.00%
     6    48    12   12     7 58.33%
Total exact: 43 / 48
Teacher-forced correct positions: 258 / 264
Proposed lengths: train [3,4], validation [5], test [6,7]; common cap 8.
```

14. Challenge: greedy or most probable?

Multiply the conditional probabilities along each complete path, including the ending. After a second ordinary token the next EOS has probability one.

Complete sequence	Probability
A EOS	$0.6(0.51) = 0.306$
A B EOS	$0.6(0.49)(1) = 0.294$
B EOS	$0.4(0.99) = 0.396$
B A EOS	$0.4(0.01)(1) = 0.004$

These four probabilities sum to one. Greedy decoding chooses A first because 0.6 exceeds 0.4, then EOS because 0.51 exceeds 0.49. Its sequence has probability 0.306. The most probable complete sequence is B EOS, with probability 0.396.

Choosing A commits to a branch whose probability is split fairly evenly between two endings. Almost all of the B branch's probability goes to one ending. A locally better first choice therefore need not belong to the globally best complete path. Greedy decoding is a selection procedure, not a guarantee of finding the maximum-probability sequence. Its output is deterministic here; the model still assigns probabilities to all four paths.

In [16]

```
paths = {"A EOS": .6*.51, "A B EOS": .6*.49,
         "B EOS": .4*.99, "B A EOS": .4*.01}
np.testing.assert_allclose(sum(paths.values()), 1.)
greedy_path = "A EOS"
best_path = max(paths, key=paths.get)
assert best_path == "B EOS" and paths[best_path] > paths[greedy_path]
for path, probability in paths.items():
    print(f"{path:9s} {probability:.3f}")
print("Greedy:", greedy_path, " Most probable complete sequence:", best_path)
```

Output

```
A EOS     0.306
A B EOS   0.294
B EOS     0.396
B A EOS   0.004
Greedy: A EOS  Most probable complete sequence: B EOS
```